

Secure Vault

Multi-user encrypted document exchange
on a server we do not trust

LAYLA

encrypts · signs

OMAR

verifies ·
decrypts

ATTACKER

reads · changes ·
replays

Shatha Abualrob 1231279 · Lara Daifallah 1230239 · Razan Shalabi 1230874

ENCS4320 Applied Cryptography · Birzeit University

02 The problem

Layla shares her thesis with Omar through a server neither of them trusts.

Users

Trusted machines.
Only they know their passwords.

Server

Honest-but-curious.
Stores everything and may be stolen.

Attacker

Controls the network.
Reads · changes · replays · registers.

Goal: the server stores everything — but learns nothing useful.

03 What must the system guarantee?

Security properties derived from the threat model.

Confidentiality

Only Layla and Omar can read the file.

Data-origin authentication

Omar can verify who sent it.

Credential protection

A stolen database should not reveal passwords.

Metadata binding

Name, type, size, time, version and ID cannot be silently changed.

Integrity

Any modification must be detected.

Non-repudiation

A third party can verify the sender.

Freshness

Old versions and replayed requests are rejected.

04 Architecture

All cryptography happens on the client. The server only stores and relays bytes.

CLIENT

Keys in memory only
Encrypt · sign · verify
No plaintext sent to server

ATTACKER PROXY

Real network position
Flip · rename · replay · swap
Used in the live demo

SERVER

Stores opaque bytes
Checks signatures + nonces
Never receives enc_key or DEKs

05 Our design decisions

1. Passwords

Argon2id

2. Documents

AES-256-GCM

3. Key agreement

X25519

4. Signatures

Ed25519

5. Key trust

TOFU + Safety Number

6. Freshness

version · nonce · challenge

7. Architecture

sockets + binary formats

8. Sessions

no tokens · signed requests

06 OUR PRIMITIVES

The small cryptographic building blocks behind SecureVault.

01

SHA-256

02

SHA-512

03

HMAC

04

HKDF

05

Argon2id

06

AES-256

07

GCM

08

X25519

09

Ed25519

07 The document is a package

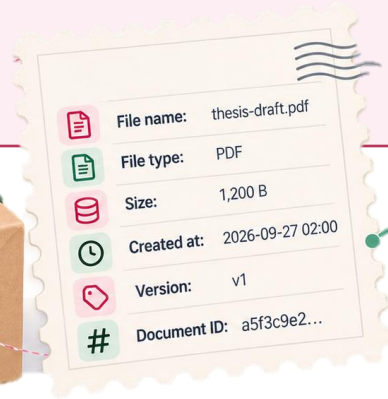
The content is sealed. The metadata stays visible — but must not be editable.

ENCRYPTED DOCUMENT

The file content lives inside the sealed package.

Encrypted file

Only authorized recipients can open the content.



Metadata

Visible information about the file, but protected from modification.

📄	File name:	thesis-draft.pdf
📄	File type:	PDF
📏	Size:	1,200 B
🕒	Created at:	2026-09-27 02:00
🏷️	Version:	v1
#	Document ID:	a5f3c9e2...

METADATA LABEL

- File name
- Type
- Size
- Time
- Version
- Document ID

Readable ≠ changeable

08 01 • SHA-256

A 32-byte fingerprint for any amount of data.

What it does

message → SHA-256 → 32-byte digest

A tiny change produces a completely different digest.

Where we use it

- Hash plaintext before signing
- Hash metadata before signing
- Build the Safety Number
- Foundation for HMAC / HKDF

Rejected: MD5 and SHA-1 are broken.

09 02 • SHA-512

Same hash-family idea, 64-byte output — required internally by Ed25519.

Why it exists here

Ed25519 (RFC 8032) uses SHA-512 internally to derive its secret scalar and deterministic per-message value.

Design role

We do not use SHA-512 as the document hash. SHA-256 remains our general fingerprint; SHA-512 serves Ed25519.

10 03 • HMAC

A hash with a secret key.

HMAC(key, message) → authentication tag

Login challenge

Server sends a fresh challenge.

Layla returns HMAC(auth_key, "SecureVault login v1" || challenge).

The password never crosses the network.

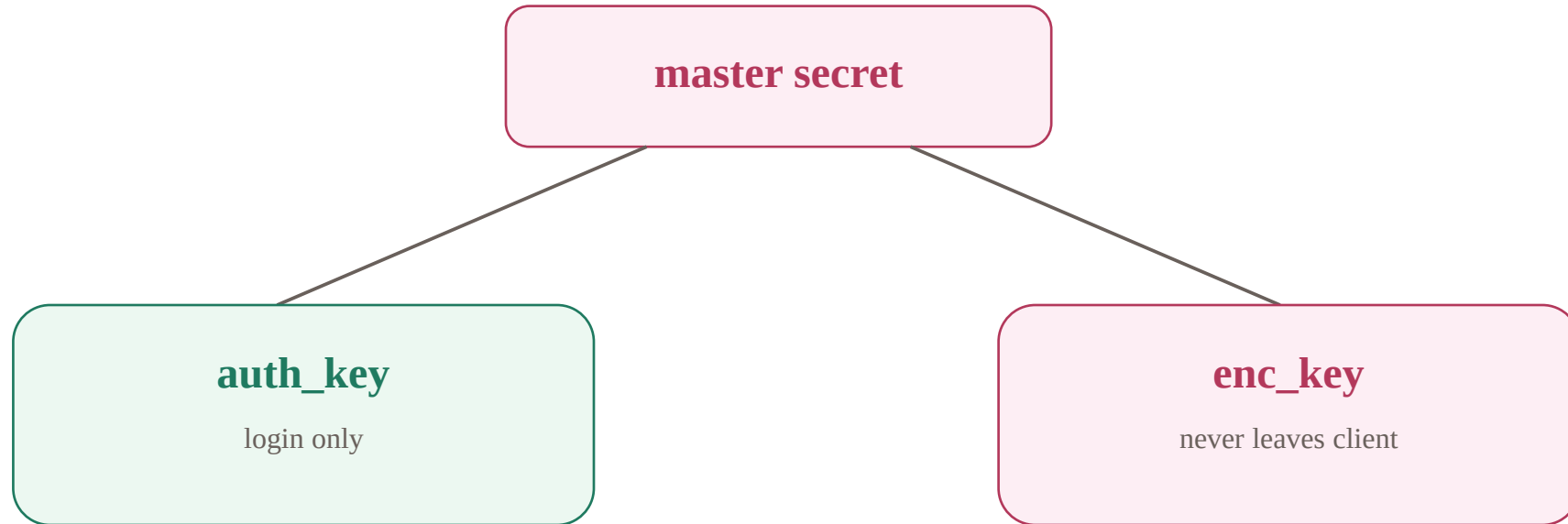
Also used for

- Fake salt for unknown usernames
- HKDF is built on HMAC
- Keyed authentication without exposing the key

11 04 • HKDF

A key factory: one secret in, independent keys out.

Same input secret + different labels → unrelated output keys



Key separation: giving the server `auth_key` must never give it the power to open the private-key blob.

12 05 · Argon2id

Make every password guess intentionally expensive.

Our parameters

384 MiB memory
3 passes · p=4
≈ 0.47 s measured

Layla

Pays the cost once per login.
≈ half a second

Attacker

Pays the same cost for every guess
+ 384 MiB memory

16-byte random salt per user · public, unique, stored beside the hash

Rejected: bare SHA-256 (too fast) · PBKDF2 (not memory-hard) · bcrypt (tiny memory / 72-byte limit) · scrypt (less independent tuning)

Project estimate: bare SHA-256 ≈ 1.4 h vs Argon2id ≈ 12,500 years for an 8-character alphanumeric search space under our assumptions.

13 06 • AES-256

Our core block-cipher primitive.

What AES does

16-byte plaintext block + 256-bit key
→ 16-byte ciphertext block
AES-256 uses 14 rounds.

Why AES-256

FIPS 197 standard
Covered in the course
Larger security margin than AES-128
Only four extra rounds.

But AES alone encrypts only one 16-byte block. A document can be megabytes.

WE NEED A MODE

14 07 • GCM

One mode, two goals: confidentiality + integrity.

GCM_encrypt(key, nonce, plaintext, AAD) → ciphertext + 16-byte tag

CTR side

Encrypts the whole document using a keystream derived from AES + nonce + counter.

GHASH side

Authenticates ciphertext and AAD. Any change makes verification fail.

AAD

Metadata stays readable, but is covered by the authentication tag.

We verify the tag before accepting any plaintext.

15 GCM's golden rule

Never reuse the same (key, nonce) pair.

$$C_1 = P_1 \oplus \text{Stream} \quad C_2 = P_2 \oplus \text{Stream} \quad \Rightarrow \quad C_1 \oplus C_2 = P_1 \oplus P_2$$

Why reuse is catastrophic

The keystream cancels out. Known plaintext can expose the other message, and GCM authentication can also be compromised.

How SecureVault prevents it

Every document version gets a fresh random 256-bit DEK.
Each DEK encrypts exactly one document version.

Crash or restart? Still safe: uniqueness does not depend on a remembered counter.

16 08 • X25519

How can Layla deliver the DEK to Omar without revealing it?

Send the DEK directly

Server and network attacker can read it.

Encrypt with a shared secret

They do not have a shared secret yet.

X25519 ✓

Both derive the same shared secret without ever transmitting it.

The shared secret exists only on the two clients.

17 X25519 is Diffie–Hellman

Same idea — elliptic-curve arithmetic.

LAYLA

Private: e

Public: $E = e \times G$

Computes: $e \times O$

Same secret S

$e \cdot o \cdot G$

OMAR

Private: o

Public: $O = o \times G$

Computes: $o \times E$

Attacker sees G , E and O — but recovering e or o from the public key is computationally infeasible.

32-byte keys · ~128-bit classical security · RFC 7748

18 X25519 in SecureVault

Ephemeral-static Diffie–Hellman wraps the DEK for Omar.

- 1 Layla creates a fresh ephemeral X25519 key pair (e, E).
- 2 She combines e with Omar's verified public key O $\rightarrow$ shared secret S.
- 3 $\text{HKDF}(S, \text{salt}=E \parallel O, \text{info}=\text{"SecureVault key grant v1"}) \rightarrow \text{wrap_key}$.
- 4 AES-GCM encrypts the grant containing the DEK + signature.
- 5 Omar uses E + his private key o to derive the same wrap_key and open it.

The ephemeral private key e is discarded after the grant is created.

19 09 • Ed25519

Digital signatures: only the private-key holder can sign; anyone can verify.

Why we need it

GCM and HMAC use shared secrets. Omar could generate the same kind of tag, so they cannot provide third-party proof.

What Ed25519 gives us

- Data-origin authentication
- Non-repudiation
- 64-byte signatures
- Deterministic per-message signing value

Private key → SIGN · Public key → VERIFY ✓ / ✗

Ed25519 uses SHA-512 internally — the reason SHA-512 appears in our primitive set.

20 Ed25519 in SecureVault

We sign the document statement and every client request.

DOCUMENT STATEMENT

"SVS1"

- + Document ID + Version
- + Sender + Recipient
- + SHA-256(Metadata)
- + SHA-256(File)

Omar reconstructs the statement himself.
Any change → verification fails.

EVERY REQUEST

- Type + Username
- + Timestamp + Nonce
- + Body

→ Ed25519 signature

Server verifies every request.
Replay attempts are rejected.
No session token is required.

The document signature is hidden inside the encrypted grant with the DEK, so the server cannot use it to test plaintext guesses.

21 MAC or signature?

We use both — each for a different security property.

MAC / GCM TAG

Shared secret key

- ✓ Detects modification
- ✓ Efficient integrity check
- ✗ Cannot prove which party created it

Used for:

- encrypted document + metadata
- encrypted grant

ED25519 SIGNATURE

Private key signs; public key verifies

- ✓ Identifies the signing key
- ✓ Third-party verifiable
- ✓ Supports non-repudiation

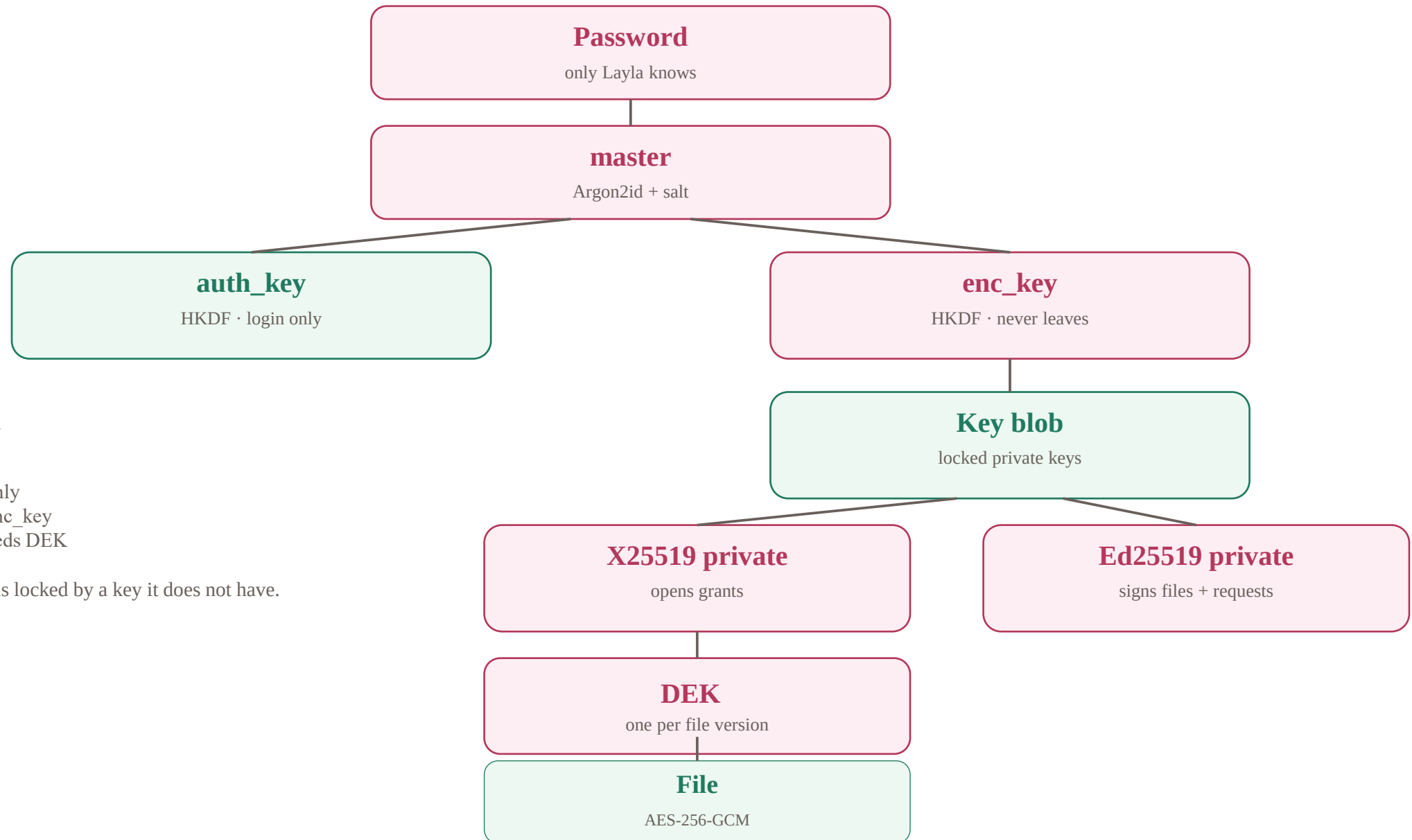
Used for:

- document statement
- client requests

A longer MAC tag still cannot give non-repudiation: Omar holds the same shared key.

22 Key hierarchy

Golden rule: each layer only unlocks the layer directly below it.



SERVER VIEW

auth_key → login only
key blob → needs enc_key
encrypted file → needs DEK

Everything it stores is locked by a key it does not have.

23 Why the hierarchy matters

A password change touches only the top of the tree.

CHANGES ✓

- New salt
- New master
- New auth_key
- New enc_key
- Re-encrypt the small key blob

DOES NOT CHANGE ✓

- X25519 private key
- Ed25519 private key
- DEKs
- Encrypted documents
- Safety numbers

24 Trusting a public key

X25519 and Ed25519 are only secure if the public key really belongs to the person.

What if the server gives Layla the attacker's key instead of Omar's?

1 • TOFU

On first contact, the public key is pinned locally.

2 • Safety Number

20 digits from both names + both keys using SHA-256.

3 • Verify

Compare by phone / in person.
Sharing stays blocked until it matches.

Changed key? Block. No “continue anyway.”

25 Full procedure · Layla shares a document

From plaintext to server-stored opaque bytes.

1

Create metadata: name · type · size · time · version · document ID

2

Generate a fresh random 256-bit DEK

3

AES-256-GCM encrypts file; metadata is AAD

4

Build the signed statement and sign it with Ed25519

5

Use Omar's verified X25519 public key to derive S

6

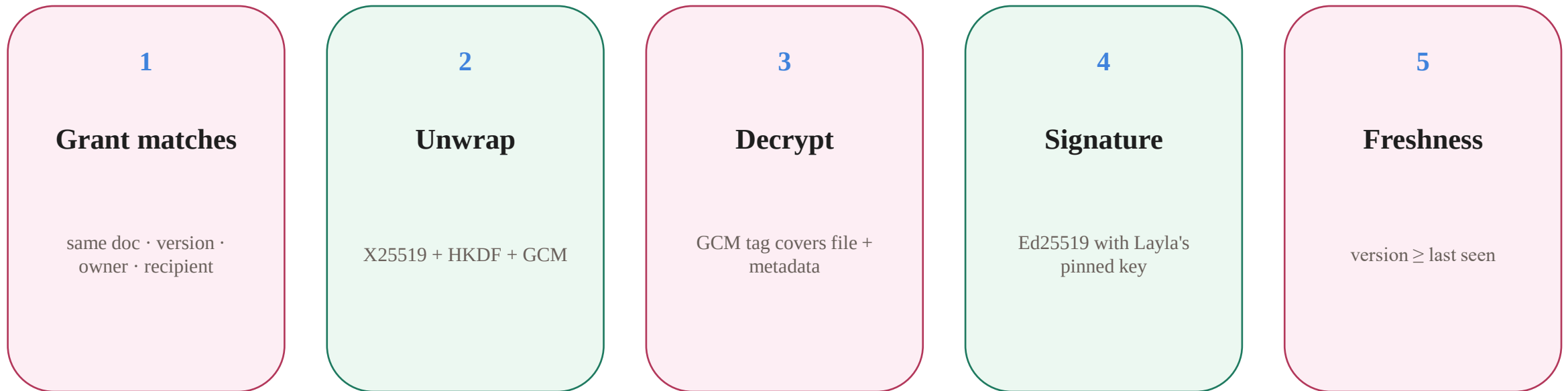
HKDF derives wrap_key; AES-GCM encrypts DEK + signature into the grant

7

Upload ciphertext + metadata + grant + Layla's ephemeral public key

26 Full procedure • Omar opens it

Five checks, in order. Nothing is shown until all pass.



Authenticate before decrypting — and before showing or saving anything.

27 Freshness & replay protection

Different layers stop different kinds of replay.

LOGIN CHALLENGE

Fresh random challenge for each login.

A captured HMAC response cannot be reused later.

SIGNED REQUEST

Timestamp + nonce are inside the Ed25519-signed request.

Captured network requests are rejected.

DOCUMENT VERSION

Omar remembers the newest version seen.

Older valid ciphertext is still rejected as stale.

28 Security properties → mechanisms

Every requirement maps to a concrete mechanism.

Confidentiality

AES-256-GCM + DEK wrapped with X25519/HKDF

Integrity

GCM tag checked before accepting plaintext

Data-origin

Ed25519 over the bound statement

Non-repudiation

Ed25519 proof verifiable by a third party

Credential protection

Argon2id 384 MiB + per-user salt

Freshness

challenge + signed timestamp/nonce + version

Metadata binding

metadata as AAD + SHA-256(metadata) in signature

29 How we tested our own cryptography

Test vectors prove the algorithm; cross-checks catch implementation mistakes.

OFFICIAL TEST VECTORS

Known inputs → known outputs

FIPS 180-4 · NIST GCM

RFC 5869 · RFC 7748

RFC 8032 · RFC 9106

If our output differs → implementation is wrong.

RANDOM CROSS-CHECKS

Compare our implementation against the cryptography library.

Hundreds of cases:

partial blocks · empty inputs · boundaries · random values

Catches edge-case bugs.

226 tests passing · system tests cover formats, keys, sharing, server and client.

We attacked our own system

A real proxy sits on the network during the demo.

Flip ciphertext byte



Document modified → rejected

Rename the file



Metadata authentication fails

Replay yesterday's v1



Stale version → rejected

Swap Omar's key



Safety numbers differ → blocked

Resend captured request



Nonce/timestamp replay → rejected

31 Why the defences are there

The same attacks succeed on a deliberately weakened build.

FIXED NONCE

Weak build:

One known plaintext can expose another because the CTR keystream repeats.

SecureVault:

Fresh DEK for every document version → unrelated keystream.

NO TAG CHECK

Weak build:

CTR is malleable; an attacker can alter ciphertext predictably.

SecureVault:

GCM authentication fails → document is rejected.

35 Live demo & questions

server · attacker · Layla · Omar

Encrypt → Share → Verify → Attack → Reject → Prove

SERVER

ATTACKER

LAYLA

OMAR

SecureVault: the server can store the system without becoming the system's trusted party.